

Universidade Federal do Rio de Janeiro - COC472

JOÃO PEDRO LEYSSIEUX CAMPANELLA

DRE: 120060835

Relatório HPC

Rio de Janeiro, 19 de junho de 2026

Exercício 1 - Variantes do 'Hello World'

1) a) *hello_os.c*

Para a abordagem deste exercício, foram utilizadas as funções *getpid()* e *sched_getcpu*. No **processo raiz (p0)**, iterou-se sobre o número total de processos e foi utilizado o método *MPI_Recv* para receber individualmente as mensagens dos outros processos. A mensagem a ser recebida era *'greeting'*, um *array* de 100 caracteres.

Nos outros processos, foi atribuído à variável *greeting* a mensagem que mostra o *rank* (processo) atual, o resultado do *getpid()* e o resultado do *sched_getcpu*. Em seguida, a mensagem com o *greeting* foi enviada ao **processo raiz (p0)**.

Ao executar *'make'* para compilar *'make run P=8'*, verifica-se:

```
jplca@Linux:/mnt/c/Users/jplca/Documents/w1---HPC/T1_leyssieux_joao/ex1_hello_os$ make run P=8
mpiexec -n 8 ./hello_os
Hello from rank 0/8 -- PID = 2228, CPU = 1
Hello from rank 1/8 -- PID = 2229, CPU = 2
Hello from rank 2/8 -- PID = 2230, CPU = 4
Hello from rank 3/8 -- PID = 2231, CPU = 6
Hello from rank 4/8 -- PID = 2232, CPU = 9
Hello from rank 5/8 -- PID = 2233, CPU = 11
Hello from rank 6/8 -- PID = 2234, CPU = 12
Hello from rank 7/8 -- PID = 2235, CPU = 14

jplca@Linux:/mnt/c/Users/jplca/Documents/w1---HPC/T1_leyssieux_joao/ex1_hello_os$ make run P=8
mpiexec -n 8 ./hello_os
Hello from rank 0/8 -- PID = 2352, CPU = 0
Hello from rank 1/8 -- PID = 2353, CPU = 2
Hello from rank 2/8 -- PID = 2354, CPU = 5
Hello from rank 3/8 -- PID = 2355, CPU = 6
Hello from rank 4/8 -- PID = 2356, CPU = 9
Hello from rank 5/8 -- PID = 2357, CPU = 10
Hello from rank 6/8 -- PID = 2358, CPU = 12
Hello from rank 7/8 -- PID = 2359, CPU = 15
```

Como observado na saída acima, ao executar o programa múltiplas vezes, nota-se que cada processo *MPI* possui diferentes *PIDs*, confirmando que o *MPI* cria processos separados na memória. Além disso, a coluna de *CPU* varia entre as execuções, o que demonstra o sistema operacional distribuindo a carga de trabalho entre os núcleos físicos disponíveis no processador.

1) b) *hello_ring.c*

Antes da lógica principal foram criadas duas funções:

- *int dest()*
- Retorna $k+1 \% p$
- *int source()*
- Retorna $(k - 1 + p) \% p$

As funções são chamadas e armazenadas nas variáveis *destination_value* e *source_value*. A seguir, a mensagem que mostra o rank, o ID do processo e CPU é atribuída à variável *greeting*. Após isso, caso o processo seja o **processo raiz (p0)**, ele apenas printa *greeting*.

Um anel com *Recv* antes de *Send* gera *deadlock*, logo que todo rank fica no aguardo de receber a mensagem do seu vizinho à esquerda. Para evitar isso, foi utilizada a função *MPI_Sendrecv_replace*. Nesta função, são os principais parâmetros o *greeting*, o tamanho máximo da string a ser enviada/recebida, o tipo do dado *MPI*, o *destination_value* e o *source_value*.

Exercício 2 - Regra do Trapézio, Generalizada

Inicialmente, foi criada a função *double f(double x)*, que retorna o valor de $\sin(x)$. Em seguida, foi criada *double Trap*, sendo os seus parâmetros:

- Ponto limite esquerdo (*double left*)
- Ponto limite direito (*double right*)
- Número de trapézios (*int n_traps*)
- Largura do trapézio (*double h*)

A função *Trap* retorna o somatório de todos os trapézios do intervalo determinado. Ademais, foi criada uma função *Get_input* que coleta os valores de **a**, **b** e **n** do usuário. Caso o *rank* atual seja o **processo raiz (p0)**, ele envia o valor de

a, b e n aos outros processos. Caso contrário, os **outros processos recebem estes valores** que foram enviados do **processo raiz (p0)**.

Para que os processos calculem a integral correta, foi calculado um *offset* para cada um, garantindo que os intervalos `[local_a, local_b]` sejam contíguos e não sobrepostos, cobrindo exatamente o intervalo `[a,b]` definido pelo usuário.

```
jplca@Linux:/mnt/c/Users/jplca/Documents/W1---HPC/T1_leyssieux_joao/ex2_trap$ make run P=
mpirun -n 3 ./mpi_trap_generalized
1
4
5
Enter a, b, n: With n = 5 trapezoids, integral of f on [1, 4] = 1.157910779844092e+00
Execution time - 30.402720 seconds
```

Exercício 3 — Soma Paralela

Foi definido manualmente o tamanho do vetor igual a 1000. Foram inicializados ponteiros *random_vector* e *local_vector* e, em seguida, alocou-se espaço para eles na memória. Quando o processo é o **processo raiz (p0)**, atribuiu-se um valor aleatório (de valor entre zero e um) para cada posição de *random_vector*.

Caso não exista resto na divisão, a quantidade de valores enviados pelo processo raiz é o valor exato da divisão $n/comm_sz$. Os valores de *random_vector* são copiados em *local_vector*.

Caso o resto da divisão não seja zero, em algum(ns) do(s) processo(s) será(ão) adicionado(s) o(s) valor(es) remanescente(s) da divisão para que a distribuição faça sentido.

Quando o processo não é o raiz, ele receberá o vetor dividido enviado por p0. Em seguida, é feita a soma local, e a soma parcial é enviada ao p0 (caso o rank atual não seja p0). Caso o rank atual seja p0, ele irá somar os valores recebidos a soma total. Por fim, é feita a comparação com o caso da soma serial (não-paralela) e são liberados os espaços de memória dos vetores.

```
jplca@Linux:/mnt/c/Users/jplca/Documents/W1---HPC/T1_leyssieux_joao/ex3_psum$ make run P=8
mpirun -n 8 ./mpi_psum
Parallel sum: 476.623366
Serial sum: 476.623366
Relative error: 4.547474e-13
```

Exercício 4 — Prática de Comunicação Coletiva

4) a) *hello_gather.c*

É criada uma variável do tipo *pid_t* para armazenar o identificador do processo. Além disso, também utilizou-se uma constante de string para definir o formato da mensagem. Em seguida, a mensagem de retorno é atribuída à variável *greeting*.

Também é criado um ponteiro (**gather_greetings*) para armazenar todas as *greetings* recebidas pelo **processo raiz (p0)**. É utilizado o *MPI_Gather* para unir todas as mensagens recebidas pelo p0 em um só vetor (*gather_greetings*). O *MPI_Gather* é uma função coletiva, logo, todos processos precisam chamá-la. No final, tudo é printado sequencialmente para o usuário.

```
jplca@Linux:/mnt/c/Users/jplca/Documents/w1---HPC/T1_leyssieux_joao/ex4_coletivas$ make run
mpiexec -n 4 ./hello_gather
Hello from rank 0/4 -- PID = 3039, CPU = 1
Hello from rank 1/4 -- PID = 3040, CPU = 2
Hello from rank 2/4 -- PID = 3041, CPU = 4
Hello from rank 3/4 -- PID = 3042, CPU = 6
```

4) b) *minmax.c*

Inicialmente, é definido o tamanho dos blocos que serão divididos do vetor original. Caso esteja-se no **processo raiz (p0)**, alocará-se valores aleatórios entre 0 e 1 em cada posição do ponteiro *randomRootVector*. Em seguida, é feita uma verificação para garantir o máximo e mínimo serial (não paralelo). É utilizado o *MPI_Scatter* para distribuir o vetor originário de *doubles* do p0 para os outros processos.

Na próxima etapa, é feita uma verificação para atribuir o máximo e mínimo locais a partir dos vetores que foram divididos a partir do original. É utilizado o *MPI_Reduce* com *MPI_Min* e *MPI_Max*.

Essa implementação assumiu que *n* é perfeitamente divisível pelo número de processos.

Exercício 5 — Soma Paralela de Vetores (Scatter/Gather)

5) a) *mpi_vecadd_gather.c*

O **processo raiz (p0)** inicializa os vetores *x*, *y* e *z*. Caso esteja-se no processo zero e exista resto na divisão $N / \text{comm_sz}$, é chamada a função *MPI_Abort*. São criados os arrays *local_x*, *local_y* e *local_z*, e cada um tem o tamanho da divisão $N / \text{comm_sz}$.

Caso esteja-se no **processo 0**, são atribuídos o valor de “*i*” a cada posição do **vetor x** e são atribuídos os valores de “ $2*i$ ” a cada posição do **vetor y**. Os arrays de *x* e *y* são distribuídos por meio do *MPI_Scatter* para os outros processos.

Em seguida, é armazenado o valor da soma $x+y$ no **vetor local_z**. Utiliza-se o *MPI_Gather* para coletar o resultado distribuído pelo *MPI_Scatter* nos outros processos.

No final do programa, o **processo raiz (p0)** compara o resultado obtido com o resultado do cálculo serial. Os espaços alocados na memória são liberados.

```
jplca@Linux: /mnt/c/Users/jplca/Documents/wl---HPC/T1_leyssieux_joao/ex5_vecadd$ make run
mpirun -n 4 ./mpi_vecadd_gather
z = x + y:
z[ 0] = 0
z[ 1] = 3
z[ 2] = 6
z[ 3] = 9
z[ 4] = 12
z[ 5] = 15
z[ 6] = 18
z[ 7] = 21
z[ 8] = 24
z[ 9] = 27
z[10] = 30
z[11] = 33
z[12] = 36
z[13] = 39
z[14] = 42
z[15] = 45
Parallel is equal to the serial calculation.
```

5) b) *mpi_vecadd_allgather.c*

O passo-a-passo desta é muito semelhante ao da letra A deste exercício. A diferença é que é utilizado o *MPI_Allgather*, que os arrays unidos chegam a todos os *ranks*. No final, o resultado obtido também é comparado com o resultado do cálculo serial.

Exercício 6 — Um Tipo de Dado Derivado MPI

6) a) *student_struct.c*

Primeiramente é criada uma struct com os parâmetros *char name[50]*, *double grade* e *int id*. É criado um array *block_lengths* que servirá para comparação futuramente. Além disso, também é criado um array ***MPI_Datatypes*** com os tipos de dado em **MPI** da struct e um ***MPI_Aint Displacements*** com o número de deslocamentos.

Neste array ***Displacements***, em cada posição dele, é chamada a função ***offsetof*** para cada parâmetro da struct. Em seguida, é criado um novo tipo MPI para a struct, em que é associado o *block_length*, *displacement* e *type* de cada parâmetro da struct.

Caso o rank atual seja o p0, é pedido para o usuário preencher o nome, nota e id no terminal. Em seguida, é feito o ***MPI_Bcast***. Só o p0 envia os dados. Os outros processos irão receber esses dados. Ao final do programa, é liberada a memória alocada para o tipo MPI.

6) b) *mpi_three_bcast.c*

O início do código é o mesmo da letra A deste exercício. A diferença é que neste exercício são utilizados 3 ***MPI_Bcast*** para transmitir cada um dos parâmetros da struct.

```
jplca@Linux:/mnt/c/Users/jplca/Documents/W1---HPC/T1_leyssieux_joao/ex6_derived$ make run
mpiexec -n 4 ./student_struct
oi
10
2
Name: oi grade: 10.0 id: 2
Process 2 name: oi grade: 10.0 id: 2
Process 1 name: oi grade: 10.0 id: 2
Process 3 name: oi grade: 10.0 id: 2
```